

An Application-Specific Approach to Energy Optimization on Android Mobile Devices

Jun Wang[✉], Member, IEEE, Karthik Rao[✉], and Handong Ye

Abstract—On current Android mobile devices, DVFS is a key power management technique. Besides CPU, memory and GPU support DVFS as well. OS modules known as governors implement algorithms that dynamically adjust operating frequency for their respective hardware components to balance performance and power. However, the stock governors do not consider application characteristics, manage power instead of energy, and operate independently of each other, thus leading to energy inefficiency. In this paper, we present an application-specific approach to energy optimization targeting Android mobile devices. In this two-step approach, we first perform offline profiling of the target app to obtain its performance and power measurements under different DVFS settings. The profiling data are then used by a feedback loop based online controller to dynamically select energy-optimal DVFS settings for the app while trying to meet a performance target. In contrast with the default governors, our approach is application-specific, manages energy, and controls both CPU and memory in unison. Tested on a few mainstream applications, our solution achieves considerable energy savings with minimal performance loss. This paper describes the design and evaluation of the system in detail with a particular focus on problems related to the robustness of the approach.

Index Terms—Android, DVFS, energy optimization, feedback control

1 INTRODUCTION

MOBILE devices are increasingly becoming an inseparable part of people's lives. In 2011, the number of smartphones shipped globally surpassed that of the personal computers for the first time. By 2019, the number of smartphone users in the world is expected to reach 2.7 billion [1]. This trend is largely due to the availability of more and more sophisticated, feature-rich software applications, which are enabled by the advancement in semiconductor and computer architecture technologies. However, batteries that power mobile devices have not seen the same rate of improvement as enjoyed by the hardware and software. Short of new technologies, the energy budget of mobile devices is becoming saturated [2]. This makes power/energy management a critical problem for mobile device hardware and software designers.

Dynamic Voltage and Frequency Scaling (DVFS) [3] is a widely adopted technique for reducing a circuit's power consumption. It reduces power by reducing the circuit's supply voltage and operating clock frequency when appropriate. DVFS is effective for two reasons [4]: (1) in general, peak performance (correspondingly, peak voltage and frequency) is only required for a small fraction of time, and (2) a CMOS circuit's power consumption is directly proportional to its clock frequency and quadratically to the supply voltage, i.e., $p \propto f \cdot v^2$.

The Linux kernel that is part of the Android OS includes two sub-systems, *cpufreq* and *devfreq*, to manage power consumption respectively for the CPU and other DVFS-enabled

components such as the main memory. Within these two sub-systems, modules known as *governors* implement algorithms that work with the device drivers to dynamically adjust clock frequencies and consequently supply voltages. These governors are typically performance-oriented. For example, a common CPU governor called *ondemand* [5] adjusts CPU frequency based on how heavily the CPU is being utilized. As soon as a threshold is crossed, the highest frequency is selected, and the frequency is lowered only if the utilization becomes lower. This kind of DVFS strategies may be reasonable for desktop and server machines, where performance is a greater concern than power consumption, and they may also work well for low-power apps on mobile devices. However, they are not suitable for those more graphics-heavy and/or power-hungry mobile apps such as gaming, video playback, video conferencing, and so on. Indeed, in our previous work [6], we pointed out that the DVFS governors found on current mainstream Android mobile devices have a few problems, including,

- 1) They are designed in a general-purpose, application-agnostic way.
- 2) They manage power, not energy.
- 3) The governors for different components of the SoC work independently, without coordination.

Because of these problems, experiments have shown that for many mobile apps, including highly popular ones, the existing governors are not energy optimal. In an effort to address the above problems, we proposed an application-specific approach to energy optimization on Android mobile devices [6]. In contrast with the stock governors, our approach achieves energy optimization in an application-specific way through coordinated control of both CPU and memory. It should be noted that for mobile apps performance is of great importance, which could be the reason why the stock

- J. Wang and H. Ye are with Huawei US R&D, Santa Clara, CA 95050. E-mail: {jun.wang2, ye.handong}@huawei.com.
- K. Rao is with the Georgia Institute of Technology, Atlanta, GA 30332. E-mail: raokart@gatech.edu.

Manuscript received 4 Feb. 2018; revised 17 Mar. 2019; accepted 26 Apr. 2019. Date of publication 30 Apr. 2019; date of current version 3 June 2020.
(Corresponding author: Jun Wang.)
Digital Object Identifier no. 10.1109/TMC.2019.2914197

governors are performance-oriented. However, instead of following the strategy of using the lowest frequency that meets the *low-level* CPU utilization demand, we believe the problem that should be addressed is the following:

Problem. Select the DVFS settings that minimize energy while meeting *application-level* performance requirement.

Clearly, this requires knowing the application-level performance requirements. Consequently, our approach is composed of two steps: offline profiling and online controlling. In the offline profiling step, we profile an app by measuring its performance and power consumption under different DVFS settings. In addition, we measure the app's performance under the default governors, which serves as the basis for the performance target. Subsequently, in the online controlling step, when the profiled app runs, a feedback-loop based controller uses the profiling data for that app to select the DVFS settings in a way that not only meets the performance target but also has the minimum energy. Experiments show that, compared with the default governors, energy savings, substantial in some cases, can be achieved with very limited performance difference.

In this paper we focus on two important problems which were not fully explored in our prior work [6] namely, the impact of input-dependence and background loads on the robustness of the controller. The most important difference between this paper and [6] is that [6] considers a single Kalman filter for the controller, whereas here we explore the utility of multiple Kalman filters and quantify the benefits in terms of additional energy savings. Specifically, this paper makes the following contributions over our prior work:

- We present an in-depth study of the problem of input-dependence and its impact on our control design (Section 5.2)
- We also present a detailed study of the impact of the background load on our approach. (Section 5.3)
- We make a case for employing application-specific performance metrics. (Section 3.1.3)
- We introduce one Kalman filter per hardware configuration to increase the controller robustness. (Section 3.2.1)
- We introduce new test cases to demonstrate the applicability of multiple Kalman filters.

Experimental results show that our *new* approach can achieve considerable energy savings with some popular mobile apps and is also robust enough to handle input or background variations.

The rest of the paper is organized as follows. Section 2 presents background and motivation for the work and also lists related work in the literature. Section 3 provides details on the design of the system, including the latest improvements. Section 4 describes the evaluation environment and characterizes the default governors' behavior with regard to the tested apps. In Section 5 we present experimental evaluation of the system, focusing on the effects of inputs and background loads. Finally, Section 6 contains our conclusions.

2 BACKGROUND AND MOTIVATION

2.1 DVFS Support in Android

Android supports DVFS through the Linux kernel it encapsulates. Specifically, two subsystems in the kernel, *cpufreq* and *devfreq*, respectively control the CPU and other DVFS-

enabled devices such as the main memory. These two subsystems are designed in very similar ways. Most importantly, both follow the principle of separation of *policy* and *implementation* and adopt a two-layer structure accordingly. In the policy layer, modules known as *governors* each represent a DVFS strategy that determines how the clock frequency should be set. The actual setting of frequency and voltage is then implemented by the device drivers.

A Linux system usually includes a few governors for the CPU (respectively the memory), with one being designated the default. Changing the governor requires root privilege. Therefore, an ordinary Android mobile device user would use the default governors exclusively.

Linux started to support DVFS with the *ondemand* CPU governor [5]. As the name indicates, this governor dynamically adjusts the CPU frequency based on the demand on the CPU. Frequency is raised when demand is high, and lowered when demand is low. *Ondemand* measures CPU load at fixed time intervals and adjusts frequency accordingly. On a Linux desktop/server, *ondemand* is usually configured as the default CPU governor.

On current Android smartphones, however, the default CPU governor is normally the one named *interactive* [7]. This governor can be regarded as a variation of *ondemand*. As its name suggests, *interactive* is designed to be more responsive to user interactions. Like *ondemand*, it determines CPU frequency based on, and only on, the level of CPU utilization. Instead of measuring CPU load periodically, it is more aggressive in raising frequency in response to load increase. When scaling up CPU speed, *interactive* will initially select a pre-defined intermediate speed known as the "hi speed" before going further up.

DVFS for memory was not supported until a few years later. In 2011, memory DVFS was still at the research stage [8], and the *devfreq* subsystem did not appear in the Linux kernel until release 3.2 in 2012 [9]. On current Android smartphone, a common default governor for the memory is *cpubw_hwmon*. This governor works in the same spirit as the *ondemand* CPU governor. It samples memory requests from the CPU at fixed intervals, and uses historic values to predict the number of accesses in the next period and selects the frequency accordingly.

2.2 Motivation

The governors described above are utility based. They make frequency adjustments solely based on the usage of the component. Clock frequency is raised as soon as the usage increases, even if this may be too aggressive. In addition, the DVFS actions are taken at very high frequency, i.e., at the scale of tens of milliseconds. Clearly this strategy favors performance over power, and is probably because it was initially devised for desktop/server type of machines, where performance is considered more important than power consumption.

The Android system inherited some governors, e.g., *ondemand*, from the desktop Linux. Although the *interactive* governor is designed with mobile workloads in mind, at its core it is still very much utility-based. We argue that the utility-based DVFS strategy is inadequate for mobile platforms. The fundamental reason is that the demands on CPU or memory at the hardware level do not always represent true application-level performance requirements. The bias towards performance, which is at the root at those governors, leads them often to select higher than necessary

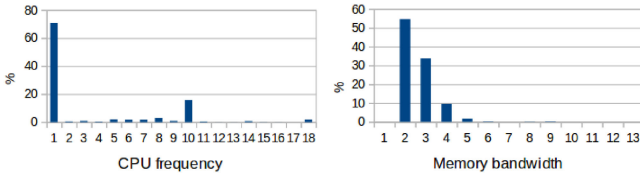


Fig. 1. Histogram of CPU frequency and memory bandwidth selected by default governors when displaying a web link content in WeChat.

frequencies when the same performance can be achieved with lower frequencies.

The inefficiency of the utility-based governors on mobile platforms has been studied before. Bezerra et al. [10] compared the power consumption of *ondemand* with simply setting the CPU frequency to a fixed value. They tested four common use cases, i.e., 3G, WiFi, voice call, and ebook reading. It was found that, in two of the four cases, *ondemand* had higher power consumption than most fixed frequencies.

To further demonstrate this, we performed a simple experiment on a Nexus 6 smartphone, a more recent device than what was used in [10]. On this phone, we ran the WeChat [11] app with no other apps running in the background. We opened a web link inside WeChat through WiFi. After the contents of the link were completely loaded, we kept the contents displayed for about 60 seconds with no user interactions, and measured the CPU and memory DVFS for that period.

In this scenario, the frame rate is 0. Obviously, using the lowest CPU frequency and lowest memory bandwidth is sufficient and most energy efficient. Fig. 1, however, shows the histograms for the CPU frequencies and memory bandwidths selected by the default governors. It can be seen that CPU frequency 10 (the “hi speed” as mentioned above) is selected over 15 percent of the time, and even the highest frequency is selected for nearly 2 percent of the time. On the memory side, the lowest bandwidth is not selected at all, while almost the entire time is distributed among bandwidths 2, 3, and 4. A similar experiment can be found in [6] as well.

This experiment shows that both the CPU and memory governors can be too aggressive, and it is possible to reduce energy consumption without performance degradation.

2.3 Related Work

Three research directions are most relevant to our work presented in this paper: (1) energy management based on application characteristics, (2) coordinated control of multiple components, and (3) control-theory based energy management.

Even before the smartphone era, researchers had realized the importance of using application level, as opposed to hardware level, characteristics for the purpose of energy management. Ellis [12] in 1999 pointed out that the application’s needs should be the driving force for the OS’ power management function. With the widespread of mobile devices, there has been a great deal of research on power management using application characteristics and/or user behavior patterns as the main guidance. In particular, web-based and frame-based applications have been most extensively studied.

Zhu et al. [13] proposed an event-based scheduler for mobile web applications that can dynamically derive user’s QoS expectations and select the optimal core and frequency settings. They claim 37 percent energy savings can be achieved compared to *interactive*. Bui et al. [14] studied the internals of web page loading and derived design principles for web browsers.

to better balance energy and performance. Nachiappan et al. [15], on the other hand, focused on frame-based applications such as video recording. They argued that by using domain knowledge such as frame rate, energy savings can be obtained with limited performance degradation. Liang et al. [16] proposed a DVFS method exclusively for video decoding, a very common operation on mobile devices. Using a two-level table structure for historic decoding performance and a global phase history table, the method predicts the complexity of the next frame and sets the CPU frequency to the lowest that can meet the decoding deadline. In a previous work [17] we explored the idea of setting up direct communication between an application and the energy manager so that application level information can be used for better energy management.

On current Android mobile devices, there are multiple hardware components that support DVFS. However, governors for these components to date have been designed to operate independently of each other. Research has shown that it is beneficial if DVFS of these components can be carried out in a coordinated manner.

Deng et al. proposed CoScale [18] to simultaneously adjust DVFS settings for both CPU and memory. Like our solution, CoScale also considers performance constraints. However, this work was carried out on a simulator and targeted a particular server workload. Our solution runs on real-world mobile devices and handles different kinds of applications. Begum et al. [19] also investigated the interplay of CPU and memory DVFS. Using a metric called *inefficiency* that they introduced, they demonstrated that the frequency of adjusting DVFS settings can be reduced with minimal performance loss, thereby reducing the overhead of the energy management system. Pathania et al. [20] studied the interdependence of CPU frequency and GPU frequency when running 3D games. Based on the analysis, they proposed a method to jointly set CPU and GPU frequencies at runtime using a pre-built performance model. In a slightly different vein, Carroll and Heiser [21] examined the interaction of CPU DVFS and core offlining and proposed a governor that integrates the control of both.

From a control-theory point of view, the default governors on current Android mobile devices can be considered as open-loop controllers because there are no evaluations of the actions taken by the governors. In contrast, researchers have explored using feedback control for energy management. Kadjo et al. [22] used closed-loop control in an energy optimizer targeting integrated CPU and GPU subsystems for graphic applications. Imes et al. proposed the POET system [23] whose goal is to optimize energy consumption of a generic embedded system while attempting to meet a performance target. To do this, first offline profiling of the target application is performed to measure its performance and power under different system configurations for resources. The source code of the application is also modified so that it can report its performance to POET. The runtime of POET uses a feedback control to select the energy optimal resource configurations based on the difference between the actual performance and the target. Inspired by this work, our previous work [6] proposed an application-specific, performance-aware, feedback-based, coordinated energy management system for Android mobile devices. Our solution was designed to address the problems of the current Android governors as listed in Section 1. In this paper, we further make important improvements to the controller to better reflect the characteristics of the target platform.

TABLE 1
Example Profiling Data for CandyCrush

(Idx, Idx)	(MHz, MBps)	Perf. (FPS)	Power (mW)
(4, 1)	(729.6, 762.0)	53.805	2393.97
(5, 1)	(883.2, 762.0)	54.465	2410.28
(4, 7)	(729.6, 1036.8)	55.781	2448.61
(5, 7)	(883.2, 1036.8)	57.768	2526.42
(6, 1)	(960.0, 762.0)	57.830	2578.95

Finally, as one of the main problems we study in this paper has to do with background apps, we briefly mention some research in this area. Chen et al. [24] performed a user study on the energy consumption by background app activities during screen-off periods and proposed a solution to manage background apps. Martins et al. [25] similarly studied how activities of background apps can be filtered or rate-limited to reduce their energy consumption. Our study of background apps, however, focuses on how they affect the overall system energy and power consumption.

3 SYSTEM DESIGN

Our solution differs from others in its application-specific approach. Specifically, the optimization process consists of two phases: app profiling and online controlling. In the app profiling phase, we profile the given app under different system configurations. The resulting application-specific profiling data are then passed to the online controller, which in turn, tries to optimize energy consumption for the app while maintaining a performance target. In the following we describe the two phases in detail.

3.1 Phase I: App Profiling

The core idea of our solution is using application-specific profiling data to optimize energy consumption of individual apps. Therefore, before DVFS control can be customized for an app, we need to collect its profiling data. In the profiling phase, we collect two kinds of data for a given app.

- Profiling table consisting of performance and power consumption of the app under different system configurations.
- Performance of the app under default governors.

The term *system configuration* refers to hardware and/or software settings of the system, for example, CPU frequency, memory bandwidth, GPU frequency, storage system parameters, OS scheduling policy, and so on. In the context of this paper, this term refers to the combination of CPU frequency and memory bandwidth. While GPU frequency may appear to be a good candidate for optimization, as is the case with CPU frequency and memory bandwidth, the effectiveness is dependent on the applications. We did not include GPU frequency because for the apps we tested, GPU DVFS did not show energy saving potentials, as illustrated in Section 4.2.

3.1.1 Profiling Data

To collect the profiling data, we set the system configuration to various values, run the app, and measure the performance and power consumption. The collected data are organized as a table with each row showing the data for one particular configuration. Table 1 shows some profiling data for the app CandyCrush [26] running on a Nexus 6 smartphone.

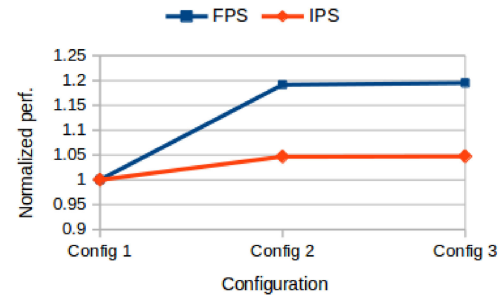


Fig. 2. Normalized performance of an app under three configurations: FPS versus IPS.

In Table 1, each entry in the first column is a pair of selected CPU frequency and memory bandwidth indices, while the second column shows the corresponding values in MHz and MBps. It is worth noting that, although the number of possible system configurations can be quite large (e.g., 234 on Nexus 6), in practice, only a very small set, often no more than 5, would suffice. For any given app, we can determine the useful range of CPU frequencies $[f_1, f_2]$ such that the performance is unacceptable below f_1 while it reaches its peak at f_2 . For memory bandwidth, we generally use the lowest value, and only select a higher value when there is a big performance gap between two neighboring configurations, e.g., (5, 1) and (6, 1) in Table 1.

3.1.2 Practical Issues

Selecting Target Apps. The target apps are those that run for more than a few seconds and use hardware resources relatively heavily. Typical candidates include applications related to gaming, video playing, GPS, and so on. For maximal impact, vendors can select those that are in widespread use.

Reducing Profiling Overhead. App profiling is time-consuming. One solution is to automate the work by running a profiler on the devices, which will start collecting profiling data when a target app runs in the foreground and send the data to a server. Additionally, research in [17] has found that multiple apps in the same category, e.g., gaming, may have very similar resource usage patterns. And in that case, those apps tend to have similar profiling data. This allows us to profile just a limited number of apps for a given category. For an app that is not profiled, a machine learning method can be used to identify a profiled one that is most similar and borrow its profiling data.

3.1.3 FPS versus IPS

Since many mobile apps are graphics-based, frames per second (FPS) of the display system is a good and a natural performance metric. However, it should not be adopted for all apps without due diligence. For example, a hardware-level metric, instructions-per-second (IPS) might be a better option for some apps (See apps used in [6]). Fig. 2 compares the IPS and FPS values of an app under three different DVFS configurations. We can see that the FPS values for configurations 2 and 3 are very close, as are the IPS values. However, between configurations 1 and 2, the difference in FPS is much greater than the difference in IPS. Incidentally, this further speaks to the application-specific approach, i.e., even the performance metric needs to be considered based on the app. As for system design, a straightforward solution is to adopt the concept of inheritance from object-oriented programming and use an abstract metric while each app has its own specific metric. In

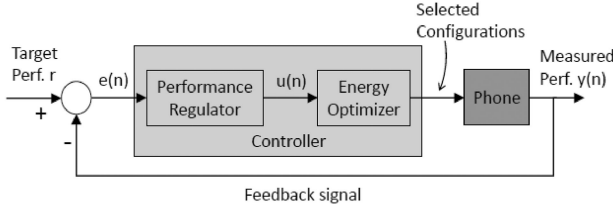


Fig. 3. Feedback loop controller.

this paper we use FPS as our performance metric because for the apps under consideration it is a more direct measurement of application level performance.

3.2 Phase II: Online Controlling

Once the profiling data and the default performance for a target app are obtained, the goal of the online controller is to use the data to achieve similar performance while minimizing energy consumption when that particular app is running. In essence, the idea is to provide customized DVFS control for the target app. Our online controller is adapted from [23] and structured as a discrete-time feedback control loop, as shown in Fig. 3.

In accordance with the dual-goal of minimizing energy while meeting a performance target, the controller is composed of two components, a performance regulator and an energy optimizer. The controller operates on a cyclic basis, and within each cycle it performs the following actions:

- 1) Measure the performance of the current cycle, $y(n)$.
- 2) Calculate the difference of the target performance and the measured performance, $e(n) = r - y(n)$. This is the input to the Performance Regulator.
- 3) The Performance Regulator computes a desired performance $u(n)$ for the next cycle. Note that this is not the same as the target performance r , because it incorporates history information. This value serves as input to the Energy Optimizer.
- 4) The Energy Optimizer, based on $u(n)$, selects from the profiling data a pair of configurations that gives the lowest energy consumption and meets the performance requirement $u(n)$.
- 5) The pair of configurations are applied for the next cycle. And the procedure is repeated.

3.2.1 (Improved) Performance Regulator

The feedback control structure adopted by our controller is a key feature that distinguishes it from the default governors, which, as described in Section 2.1, do not use any feedback at all. The feedback control function is implemented in the Performance Regulator (PR). As shown in Fig. 3, the input to the PR is $e(n) = r - y(n)$, the difference between the target performance and the measured performance, while the output $u(n)$ from the PR is a control signal that represents a desired performance level. Using feedback control, the PR can dynamically adjust the desired performance. Its goal is to reduce $e(n)$ to zero, or in other words, to achieve the performance target.

Given that our approach relies on the static offline profiling data, a great challenge is the fact that the runtime conditions may deviate from those under which the offline profiling data are acquired. Such differences must be accounted for dynamically. In our previous work [6] we adopted the performance

model from POET [23], which defines performance as $speedups(n)$ times *base speed*: $u(n) = s(n) \cdot b(n)$.

The base speed is the speed of the application when it uses the minimum amount of resources. In each cycle the desired speedup $s(n)$ is being determined, while the base speed $b(n)$ is adjusted dynamically to capture the runtime variations from the offline profiling environment. One problem with this model is that the ratio of the performance estimates of any two configurations remains constant. For example, suppose that two configurations have the speedups of s_1 and s_2 respectively in offline profiling. At runtime, their performance estimates are respectively $b(n) \cdot s_1$ and $b(n) \cdot s_2$. While $b(n)$ may change from one cycle to the next, the ratio of the two performance estimates remains unchanged. For the embedded systems POET was designed for, this is probably not a problem because the hardware and software are fixed and the performance ratio between any two system configurations is more or less constant. However, since we target Android mobile devices, which have many more unpredictable runtime variations, a different performance model is required.

For a set C of profiled system configurations, denote by r_i the profiled performance for configuration $c_i \in C$, and $N = |C|$. Our runtime performance prediction model for each c_i is as follows:

$$\hat{r}_i(n+1) = a_i(n) \cdot r_i, \quad i \in \{1, 2, \dots, N\}, \quad (1)$$

where $a_i(n)$ is a scaling factor for configuration c_i and $\hat{r}_i(n+1)$ is the predicted performance for c_i for the next cycle. For each $a_i(n)$ we use a separate Kalman filter [27] to estimate its value dynamically. Details are presented in Section 3.2.3.

To compute the desired performance $u(n)$, the Performance Regulator uses integral control, a reduced form of the proportional-integral-derivative (PID) control [28]. The control function is shown in Equations (2) and (3).

$$e(n) = r - y(n) \quad (2)$$

$$u(n) = u(n-1) + g(n) \cdot e(n). \quad (3)$$

The integral part of the control function refers to the integral of past errors $e(k)$. It can be easily derived from Eqn. (3) that $u(n) = u(0) + \sum_{k=1}^n g(k) \cdot e(k)$. Therefore, $u(n)$ contains the integral (i.e., sum in the case of discrete-time systems) of past errors. The factor $g(n)$ in Eqn. (3) is the *integral gain* and can be either a constant or a variable. In this work we use fixed gain and set $g(n)$ to a constant $g, 0 < g < 1$.

3.2.2 Energy Optimizer

Once the Performance Regulator determines the desired performance level $u(n)$, it is passed as input to the Energy Optimizer, whose goal is to find a set of DVFS configurations and corresponding application times such that the performance level is achieved with minimal energy. The term “application time” refers to the duration of time when the system is set to a particular DVFS configuration. As in POET [23], this problem is formulated as an integer linear programming problem below.

For the given set C of system configurations in the profiling table, find, for each configuration c_i , the application time $\tau_i, i = 1, \dots, N, N = |C|$, that will

$$\text{minimize} \quad \sum_{i=1}^N \tau_i \cdot p_i \quad (4)$$

$$\text{subject to } \sum_{i=1}^N \tau_i \cdot \hat{r}_i(n) = u(n) \cdot T \quad (5)$$

$$\sum_{i=1}^N \tau_i = T, \quad 0 \leq \tau_i, \quad i = 1, 2, \dots, N, \quad (6)$$

where τ_i is the application time of configuration c_i , p_i is the profiled power consumption of c_i , $\hat{r}_i(n)$ is the predicted performance for c_i as given in Eqn. (1), $u(n)$ is the desired performance, and T is the control cycle period.

Eqn. (4) is the objective of the linear programming problem, which is to minimize the energy consumption of the selected configurations. Eqn. (5) is the constraint that the performance level $u(n)$ be met, while Eqn. (6) requires the sum of the individual application times be equal to the control cycle period.

According to the theory of linear programming [29], for a linear programming problem whose constraints are expressed as $\mathbf{Ax} = \mathbf{b}$, where $\mathbf{A}$ is an $m \times s$ ($m \leq s$) matrix with rank m , the basic solutions to the problem are in the form $\mathbf{x} = (0, 0, \dots, 0, x'_1, x'_2, \dots, x'_m)$ with m non-zero values and $s - m$ 0's. Therefore, for our problem, we only need to find two non-zero values for τ_i , which simplifies the problem to the following:

$$\text{minimize } \tau_l \cdot p_l + \tau_h \cdot p_h \quad (7)$$

$$\text{subject to } \tau_l \cdot \hat{r}_l(n) + \tau_h \cdot \hat{r}_h(n) = u(n) \cdot T \quad (8)$$

$$\tau_l + \tau_h = T, \quad 0 \leq \tau_l, \tau_h, \quad 1 \leq l, h \leq N. \quad (9)$$

In other words, to solve the problem expressed in Equations (4), (5), and (6), in each control cycle we only need to find two system configurations c_l and c_h , that satisfy Equations (7), (8), and (9). Specifically, this is carried out in the following steps:

- 1) Divide the set C into two disjoint subsets C_l and C_h , respectively referred to as the *low set* and the *high set* because all the configurations in C_l has lower performance than $u(n)$ and all those in C_h has performance higher than $u(n)$.
- 2) From all the pairs (c_i, c_j) , $c_i \in C_l$ and $c_j \in C_h$, find the pair (c_l, c_h) that has the lowest energy as computed in Eqn. (7) and satisfies Equations (8) and (9).

Once the pair of configurations (c_l, c_h) are identified, we set the system to configuration c_l for τ_l and c_h for τ_h time units. At the end of the control cycle, the process described above is then repeated.

3.2.3 Runtime Adaptation

As mentioned above, it is of great importance that we properly deal with any runtime performance deviation from profiled performance caused by the difference of the runtime environment from the offline profiling environment. This is achieved with the scaling factor $a_i(n)$ in Eqn. (1). A Kalman filter is used to estimate $a_i(n)$ dynamically.

Specifically, the estimation is done in two steps: time update and measurement update [27]. A brief summary follows, and the reader is referred to [27] for more details on Kalman filter. For the sake of clarity, we omit the subscript i in the notation symbols unless it is necessary. But it should be understood that what follows is just for one Kalman filter, and there is one for each configuration in the profiling table.

In the first step, the so called *a priori* estimates for $a(n)$ and the error covariance $p(n)$ are obtained, as follows:

$$a^-(n) = a(n-1) \quad (10)$$

$$p^-(n) = p(n-1) + Q. \quad (11)$$

This is followed by the computing of the *Kalman gain* $K(n)$, and finally the *a posteriori* estimates that incorporate the latest measurement, as follows.

$$\tilde{y}_i(n) = y_i(n-1) - (a^-(n) \cdot r_i) \quad (12)$$

$$K(n) = \frac{p^-(n) \cdot r_i}{r_i^2 \cdot p^-(n) + R} \quad (13)$$

$$a(n) = a^-(n) + K(n) \cdot \tilde{y}_i(n) \quad (14)$$

$$p(n) = (1 - K(n) \cdot r_i) \cdot p^-(n). \quad (15)$$

In Eqn. (12) $y_i(n-1)$ is the measured performance for configuration c_i . The symbols Q and R above are the *process noise covariance* and the *measurement noise covariance*, respectively, and are set to constants close to 0 in our implementation. Note that, although there is one Kalman filter for each configuration in the profiling table, as described in Section 3.2.2 no more than two configurations are used in each cycle. Therefore, at most two Kalman filters are updated in each cycle.

3.2.4 Summary of Online Controlling

We summarize the actions of the online controller in Algorithm 1.

Algorithm 1. Online Controller Algorithm

repeat

Step 1: Performance Regulator:

obtain measured performances $y_l(n)$, $y_h(n)$, and $y(n)$
 update $a_l(n)$ and $a_h(n)$ with Eqns (10), (11), (12), (13), (14), and (15)
 update $\hat{r}_l(n)$ and $\hat{r}_h(n)$ with Eqn (1)
 compute control error $e(n) = r - y(n)$
 compute desired performance $u(n)$ with Eqn (3)

Step 2: Energy Optimizer:

use $u(n)$ to divide C into C_l and C_h
for all pairs (c_i, c_j) , $c_i \in C_l$, $c_j \in C_h$ **do**
 compute τ_i, τ_j with Eqns (5) and (6)
 compute energy with Eqn (4)
 store the values with minimal energy in $(c_l, c_h, \tau_l, \tau_h)$

end for

Step 3: apply c_l for τ_l and c_h for τ_h

until end

4 EXPERIMENTAL SETUP

This section describes our test environment, including the test device, the software/hardware tools, and the selected apps.

4.1 Device and Tools

All our experiments are conducted on a mainstream commercial smartphone—the Motorola Nexus 6. This device is

TABLE 2
CPU Frequencies and Memory Bandwidths on Nexus 6

CPU Frequency (GHz)				Mem Bandwidth (MBps)			
1	0.3000	10	1.4976	1	762	10	8,056
2	0.4224	11	1.5744	2	1,144	11	10,101
3	0.6528	12	1.7280	3	1,525	12	12,145
4	0.7296	13	1.9584	4	2,288	13	16,250
5	0.8832	14	2.2656	5	3,051		
6	0.9600	15	2.4576	6	3,952		
7	1.0368	16	2.4960	7	4,684		
8	1.1904	17	2.5728	8	5,996		
9	1.2672	18	2.6496	9	7,019		

TABLE 3
CPU, Memory, and GPU Usage Under Default Governors and No Background Load

App	CPU util	Mem access/s ($\times 1024$)	GPU util
Candy	29.00%	28,454	49.77%
MxPlayer	45.55%	102,198	13.19%
WeChat	56.30%	65,135	10.23%

equipped with a Qualcomm Snapdragon 805 chipset, which includes a quad-core Krait 450 CPU, 3 GB RAM, and an Adreno 420 GPU. The CPU, memory, and GPU are all DVFS-enabled. Table 2 lists the available CPU frequencies and memory bandwidths.

In order to be able to perform DVFS on the CPU and the memory bandwidth from the user space, we replace the default OS on the Nexus 6 with a custom built Android 6.0.0 along with Linux V3.10. An alternative is to simply root the phone. Once root access is available, to set CPU frequency or memory bandwidth to a fixed value, it involves first setting the corresponding governor to *userspace* and then setting the frequency/bandwidth to one of the supported values. All this is done by writing to files in the *sysfs* [30] filesystem that are exposed to the user space by *cpufreq* and *devfreq*.

Tasks such as performance measurement are performed with the help of Android services so that the source code of the framework is not modified. Power consumption of the device is measured by physically connecting the device to a Monsoon Power Monitor [31]. This is a commonly used power measuring instrument with a sampling rate of 5 KHz. It should be pointed out that the power consumption we measure is that of the entire device.

4.2 Applications

We have selected three popular Android applications respectively from three different categories for evaluation purposes. We describe each of the apps below.

CandyCrush [26] is a popular game app with over 500 million installs. We test this app by manually playing the game for about 60 seconds each time.

MxPlayer [32] is a top video playing app capable of handling various video formats. We test this app by playing high definition video files stored on the device.

WeChat [11] is a highly popular personal communication app with 963 million active users [33] as of the second quarter of 2017. In addition to text messaging, WeChat supports video chat, audio chat, and some social networking functionalities. In this paper, we test the video chat feature of

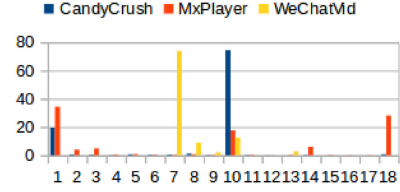


Fig. 4. CPU frequency usage histogram.

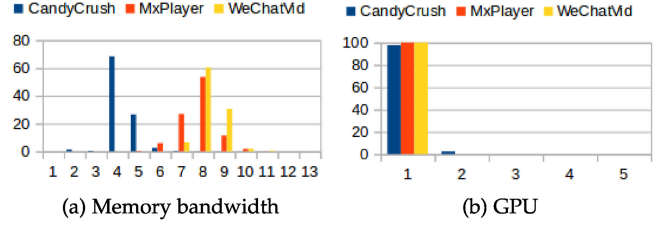


Fig. 5. Memory bandwidth and GPU frequency histogram.

WeChat by making video calls between two smartphones for about 60 seconds each time.

To better understand the behavior of these apps and to illustrate their difference, we have tested the apps and measured their CPU, memory, and GPU usage. These tests are performed under the default DVFS governors, i.e., *interactive* CPU governor and *cpubw_hwmon* memory governor. In addition, there are no other apps running in the background during the tests.

Table 3 lists the results for CPU utilization, memory access rate, and GPU utilization, averaged over three runs. The three apps clearly demonstrate different resource usage patterns. The game app CandyCrush is the least CPU-intensive, probably because this game is played by user making repeated moves with long (for the CPU) pauses between moves. CandyCrush is also the least memory-intensive, but its GPU utilization is much higher than the other two apps. MxPlayer is highly input-dependent. With the video file used in this test, it has much higher CPU utilization than CandyCrush. Its memory access rate is much higher than the other two apps, but its GPU utilization is rather low, close to that of WeChat. WeChat Video has the highest CPU utilization in the group. Its memory access rate is between the other two apps. However, among the three apps, it has the lowest GPU utilization.

A cursory examination of these results suggests that it is really challenging to create an energy manager that can perform well for different types of apps. And it appears that DVFS of the different components requires a more holistic approach.

To further illustrate the different characteristics of the three apps and how the default governors behave towards these apps, in Figs. 4 and 5 we show the CPU frequency, memory bandwidth, and GPU frequency histograms in the same tests as above. Note that in all three cases, CPU frequency No. 10, i.e., the “hi speed” for the *interactive* governor, it used for over 10 percent of the time. For CandyCrush it is used over 70 percent of the time due to the interactive nature of the app. The rest of the CPU time is mostly spent in frequency No. 1. For MxPlayer, CPU time is mostly divided among three frequencies: Nos. 1, 10, and 18. This looks like a case of oscillation caused by over-correction on the part of the *interactive* governor. The nearly 30 percent of time in the highest frequency is significant and is the major source for optimization opportunities. For WeChat, CPU time is mostly distributed among Nos.

TABLE 4
Performance, Power, and Energy Savings by Controller
Under Base Load

App	Def Perf	Perf	Perf%	Def Pwr	Pwr	Engy%
Candy	55.96	55.40	-1.0%	2648.80	2425.39	-8.4%
MxPlayer	23.98	23.82	-0.7%	3249.12	2522.26	-22.4%
WeChat	18.42	17.79	-3.4%	3727.65	3338.47	-10.4%

Target FPS are: 56 (Candy), 23.9 (MxPlayer), and 18 (WeChat).

7, 9 and 10, with over 70 percent in No. 7. It is clear that in all three cases, CPU frequency selection is concentrated on a small subset of the available frequencies.

The same is true with memory bandwidth. As shown in Fig. 5a, for each app, the default governor selects two or three bandwidths out of the 13 most of the time. In addition, those that are used most often are in the middle of the range of bandwidths—those that are below No. 4 or above No. 9 are used very little if at all.

GPU histograms are more homogeneous. For MxPlayer and WeChat, 100 percent GPU time is spent in frequency No. 1, while CandyCrush also spends 97.6 percent of time in frequency No. 1 with the rest in No. 2.

From the above test results we can see clearly that the three apps all have different requirements on CPU and memory, causing the default governors to respond in different ways. However, as we shall present in the next section, the DVFS actions taken by the default governors are generally too aggressive, making it possible to achieve energy savings with minimal performance impact.

5 EVALUATION

This section presents evaluation of our approach using the selected apps listed in the previous section. The focus here is on two problems that were not fully explored before in order to assess the robustness of the solution. Unless explicitly noted, test results are generally the average of multiple runs. It is also important to note that the experiments span many days. Even when the test environments appear to be the same, day-to-day variations are inevitable. This could cause test results to appear inconsistent occasionally.

5.1 Results for Baseline Load

We first present test results in a background load we shall refer to as the *Base Load* which we use as a baseline. Specifically this consists of running Facebook and Yahoo Email in the background. We collected offline profiling for the apps, including measuring the performance and power by the default governors. Then we ran the apps with the controller running in the background.

For CandyCrush, we played level 5 of the game for about 60s in each test run. For MxPlayer, we played an excerpt of an action movie for about 60s in each run. The video file is in the QuickTime File Format [34] (extension .mov) with a frame rate of 24, a resolution of 1920×1080 , and a video bit rate of 4.7 Mbps. For WeChat Video, we placed video calls between two phones for about 60s in each run.

In Table 4 we show the performance, power, and energy savings results of our controller in comparison with the default governors. It can be seen that the performance achieved by the controller is very close to the default, with the worst case being a 3.4 percent performance loss seen with WeChat Video. Even in that case, the 17.79FPS achieved versus the default governors' 18.42FPS should make no difference visually to a user. Furthermore, the achieved FPS is only 1.2 percent less than the 18FPS target. At the same time, the controller achieved 8.4–22.2 percent energy savings compared with the default settings. The experimental data strongly support our proposition that a DVFS strategy customized for a specific application can save energy with minimal performance impact.

In Figs. 6 and 7 we compare the CPU frequency and memory bandwidth histograms between the default governors and our controller. It is clear that, for all three apps, the CPU frequencies used by the controller are lower than those selected by the default governor most of the time. In particular, in the MxPlayer case, the default governor selected the highest frequency 26.9 percent of the time, while the controller used two much lower frequencies, directly contributing to the 22 percent energy savings.

On the memory side, the controller used the lowest bandwidth exclusively in all three cases, while the default governors used much higher bandwidths. The behavior of the controller is based on the offline profiling. In general we have

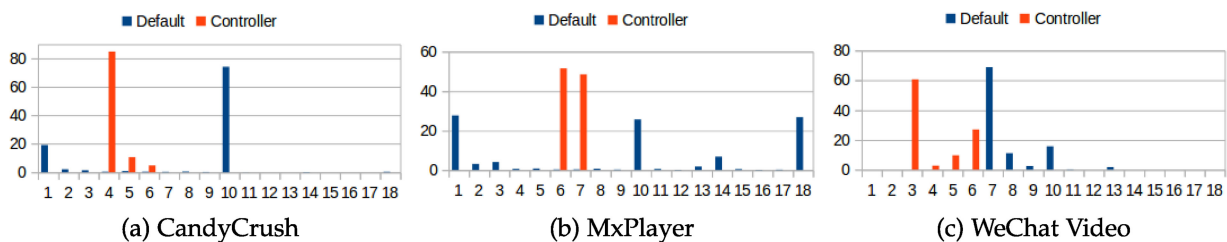


Fig. 6. CPU frequency histogram comparison.

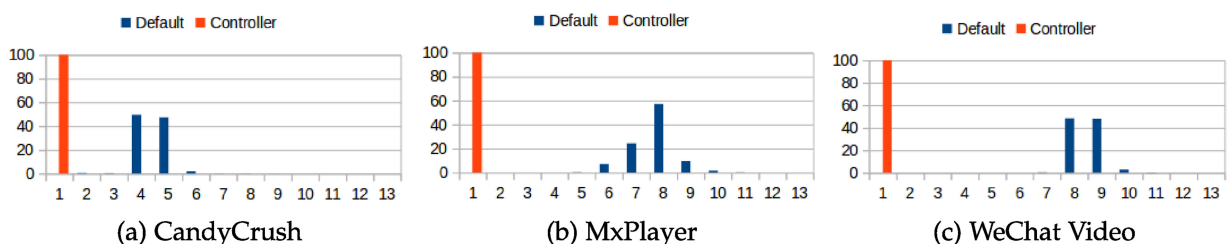


Fig. 7. Memory bandwidth histogram comparison.

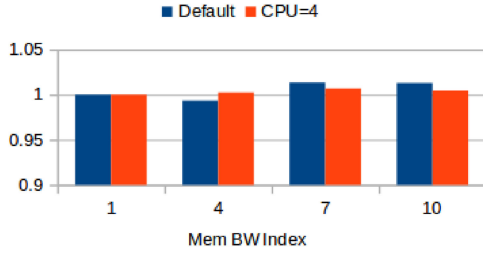


Fig. 8. CandyCrush normalized FPS with four different memory bandwidths and CPU using default governor and frequency No. 5.

TABLE 5

CandyCrush Level 5: Comparison of Performance and Power of Two Different User Interaction Frequencies

Config.	Performance			Power		
	Norm.	NoMove	Perf%	Norm.	NoMove	Pwr%
Default	55.86	52.00	-6.9%	2654.02	2451.32	-7.6%
(4, 1)	54.92	55.10	0.3%	2448.23	2417.21	-1.3%
(5, 1)	57.01	56.37	-1.1%	2485.26	2438.74	-1.9%
(6, 1)	57.71	57.86	0.3%	2569.48	2511.34	-2.3%

found that a configuration with a certain CPU frequency and lowest memory bandwidth bw_1 has a better power-performance balance than a configuration with a lower CPU frequency but a higher memory bandwidth. Sometimes the latter may even have a higher power but a lower performance.

In Fig. 8 we show the FPS values when playing CandyCrush while fixing memory bandwidth to four different settings. Two cases are shown: using default CPU governor and fixing CPU frequency to No. 5. And the FPS values are normalized to the case where memory bandwidth is set to No. 1. In both cases we can see that the FPS differences are generally minimal, even though bandwidth No. 10 (8056 MBps) is over ten times of bandwidth No. 1 (762 MBps).

5.2 Effects of Inputs

The effects of the applications' inputs are an important problem for our approach because input-dependent behavior could pose a serious challenge to the robustness of the method. In this section we examine this problem in detail. Specifically, we focus on the following research question.

RQ1. How robust are the profiling data and the general approach when subjected to various app inputs?

Obviously, each app has its own form of inputs, and its own level of sensitivity to the variations of inputs. Therefore, we look at each app separately. Additionally, an app can have many factors that can be considered as inputs. We do not attempt to exhaustively examine all possible factors, but instead focus on the most prominent variables.

5.2.1 CandyCrush

For CandyCrush, we consider two major dimensions of input variations. One dimension is the frequency of user interaction. Depending on the user, the level of the game, and other factors, the number of moves made by the user per unit time can be highly varying. Intuitively, this could have a significant impact on performance and power consumption. Another dimension is the level of the game. Different levels have different goals, scenes, game configurations, and so on, which could result in difference in performance and power.

TABLE 6

Perf. and Energy Results by Controller: CandyCrush NoMove with Normal Profiling Data and Two Different Targets

Target	Def Perf	Perf	Perf%	Def Pwr	Pwr	Engy%
54	52.00	56.46	8.6%	2451.32	2441.06	-0.4%
56	52.00	56.70	9.0%	2451.32	2449.12	-0.1%

TABLE 7

CandyCrush: Comparison of Performance and Power of Two Different Game Levels

Config.	Performance			Power		
	Lev 5	Lev 13	Perf%	Lev 5	Lev 13	Pwr%
Default	53.67	53.76	0.2%	2587.71	2647.43	2.3%
(4, 1)	53.24	53.63	0.7%	2508.69	2507.00	-0.1%
(5, 1)	55.73	54.60	-2.0%	2545.10	2561.55	0.6%
(6, 1)	57.06	57.31	0.4%	2641.73	2703.65	2.3%

Frequency of User Interaction. We tested two cases with different user interaction frequencies while playing level 5 of the game-*Normal* is when we play the game normally with about 12 moves per minute, while *NoMove* is the extreme case where we make no moves during the entire test period. Although it is unlikely to occur in reality, *NoMove* can help us understand the impact of user interactions. In Table 5 we compare the performance and power of the two cases under four different system configurations, including the default.

We can see that with the default DVFS governors, the two cases have noticeable difference in both performance (6.9 percent) and power (7.6 percent). However, when the CPU frequency and memory bandwidth are fixed, as in the other three configurations, it appears that user input has very limited impact on either performance or power. This suggests that the profiling data obtained in one case can be used in another. However, the complete absence of user interactions in the *NoMove* case means the default governor will spend much less time in the "hi speed" frequency No. 10 as it does in the *Normal* case shown in Fig. 6a—16 percent versus 74 percent. This directly leads to much less power consumption. Therefore, in this extreme case, although the profiling data are very much valid, it is very difficult for the controller to achieve lower power.

We tested the controller in the *NoMove* case using profiling data for *Normal* and two different performance targets: 54 and 56FPS. The results are listed in Table 6. Despite the fact that the target given to the controller is much higher than the FPS achieved by the default governors, the power consumption is on par with the default settings.

Game levels. CandyCrush has many different levels. In Table 7 we compare the performance and power of levels 5 and 13. Level 13 is much more difficult than level 5. While we can play around 12 moves per minute in level 5, we can only play about 7 moves per minute in level 13. To minimize the impact of the move frequency, we played level 5 at about 7 moves per minute in this experiment.

The results in Table 7 show that the two levels have very close performance and power consumption in all the four configurations. This means that we can use the profiling data obtained in one level for a different level.

TABLE 8

Perf. and Energy Results by Controller: CandyCrush Level 13 with Level 5 Profiling Data and Two Different Targets

Target	Def Perf	Perf	Perf%	Def Pwr	Pwr	Engy%
54	54.40	55.10	1.3%	2697.27	2552.70	-5.4%
56	54.40	55.54	2.1%	2697.27	2596.54	-3.7%

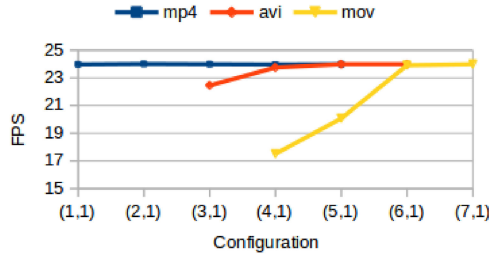


Fig. 9. FPS Under Different System Configurations: MP4, AVI, and MOV.

Finally, in Table 8 we show the controller results for playing level 13 using the profiling data from level 5. Two different targets were used: 56 and 54 FPS. The results show that energy saving of 5.4 and 3.7 percent were achieved with limited performance difference.

We conclude that, for CandyCrush, the controller is robust in the face of input variations. The profiling data obtained in level 5 with normal play remain effective when user interaction frequency or game level are different.

5.2.2 MxPlayer

In terms of input variations, video players such as MxPlayer probably have more variables than the other two types of apps we study in this paper. Such variables include file format, resolution, frame rate, bit rate, content, and so on. In this part of the experiments, we focus on two parameters, namely, file format and video content.

Video file container format. MxPlayer supports many different file container formats such as MOV, MP4, AVI [35]. We use the same video file in Section 5.1, which is in the MOV format, and convert it to the MP4 and AVI formats, leaving all the other parameters such as frame rate unchanged. We then compare the performance and power characteristics of the different file formats when played with MxPlayer.

As it turns out, the three formats have significantly different behavior. Under the default governors, while the MOV file consumes over 3200 mW power as shown in Section 5.1, the MP4 and AVI files consumes significantly less: 1522 and 2148 mW respectively.

In Fig. 9 we compare the performance characteristics of the three file formats under 7 different system configurations ordered by CPU frequency.

With the MP4 file, close to 24FPS is achieved with the lowest CPU frequency. Increasing the CPU frequency has no effect on the frame rate. On the other hand, with the AVI file, 24FPS is achieved only when the CPU frequency is increased to No. 5, while the MOV file does not reach this FPS level until CPU frequency No. 6.

Clearly in this situation different profiling tables are required for the controller to perform well. For example, for MP4 the best option is probably simply choosing the lowest

TABLE 9

MxPlayer: Comparison of Performance and Power of Two Different MOV Files

Config.	Performance			Power		
	Action	Cartoon	Perf%	Action	Cartoon	Pwr%
Default	23.95	23.91	-0.2%	3202.30	2964.05	-7.4%
(5, 1)	20.07	23.86	18.9%	2381.82	2369.36	-0.5%
(6, 1)	23.91	23.95	0.2%	2500.60	2427.05	-2.9%
(7, 1)	23.98	23.96	-0.1%	2518.03	2489.11	-1.1%

TABLE 10

Performance and Energy Results by Controller: *Cartoon* with Profiling Data for *Action*

Def Perf	Perf	Perf%	Def Pwr	Pwr	Engy%
23.94	23.95	0.0%	2988.44	2426.53	-18.8%

Target 23.9FPS.

configuration, while for MOV, CPU frequency lower than No. 5 probably should not even be included in the profiling table. However, this does not necessarily mean each file format needs a separate table, because some file formats, for example, MP4 and MKV, have very similar characteristics.

This leaves the question of how to select the correct profiling table for the controller. One possible solution is to set up a direct communication channel between the application level and the energy manager, as proposed in [17]. Through this channel application level information that is useful for the purpose of energy optimization, for example, video file format, can be passed to the energy manager.

Video contents. Even when other parameters such as file format are the same, the contents of a video file could cause differences in performance and power. To study this problem we tested two different video files: the one from Section 5.1, which is an action movie, and a cartoon that has the same frame rate and same resolution.

Table 9 shows the performance and power of the two MOV files played with MxPlayer under four system configurations. The 7.4 percent difference in power under the default governors and the 18.9 percent difference in frame rate under configuration (5, 1) suggest that the difference between the two files are significant. However, we note that, under fixed configuration, the power difference of the files are very small, and the performance are essentially the same when CPU frequency is at No. 6 and beyond.

We tested the controller with *Cartoon* using the profiling data for *Action*. The results are shown in Table 10.

On the other hand, using the profiling data for *Cartoon* to test *Cartoon* we achieved 20.3 percent energy savings with a performance loss of 0.3 percent. We can see that the results achieved by using the profiling data for *Action* are very close.

In this case, we are particularly interested in if the performance of the configuration (5,1) in the profiling table, which is 18.9 percent lower than the true value, can be corrected at runtime using the performance model and Kalman filter as detailed in Section 3.2. Recall that the performance prediction model is $\hat{r}_i(n+1) = a_i(n) \cdot r_i$, where r_i is the profiled performance, and the scaling factor $a_i(n)$ is being estimated with a Kalman filter.

Fig. 10 illustrates the runtime estimation of the performance of (5,1) in one controller run. Initially, this configuration was

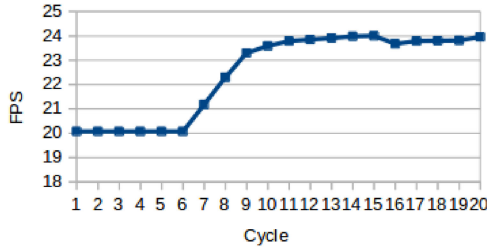


Fig. 10. Configuration (5,1) runtime adaptation.

TABLE 11
WeChat Video: Comparison of Performance and Power of Two Different Motion Levels

Config.	Performance			Power		
	Normal	NoMot.	Perf%	Normal	NoMot.	Pwr%
Default	17.33	18.15	4.7%	3638.42	3592.70	-1.3%
(3, 1)	17.15	17.21	0.3%	3255.17	3195.77	-1.8%
(4, 1)	17.31	17.54	1.3%	3301.73	3205.88	-2.9%
(5, 1)	17.79	17.69	-0.6%	3416.36	3294.11	-3.6%

not selected due to the low performance in the profiling table. At the end of cycle 6 it was selected for the first time, and the performance estimate was updated from 20.07 to 21.18, 22.30, 23.30 in the next three cycles before it settled near 23.90, which is near the true value.

The controller in this case has demonstrated its ability to adapt to runtime variations to address the discrepancy with the profiling data, which is crucial to achieving good results. One open question here, however, is to ensure that a configuration whose profiled performance is much lower than its actual runtime performance can be selected by the controller. We leave this for future work.

To summarize, in the case of MxPlayer, the behavior differences caused by file formats are too substantial for the controller to handle successfully. While some file formats may have similar characteristics and can probably share a profiling table, for those that are too different, different profiling tables must be used. At runtime, either a notification mechanism as suggested in [17] or some other technique must be utilized to help the controller select the correct profiling table.

On the other hand, for the same file format, our approach can handle the variations caused by video contents successfully. Despite the fact that for some configurations the actual performance can differ greatly from the profiling table, the runtime adaptation of our controller can rapidly converge to the true value, allowing the model in Eqn. (1) to reflect the actual performance, leading to good results.

5.2.3 WeChat Video

For WeChat Video, we study the impact of inputs by changing the degree of motion in the video in two test cases. In the *Normal* case, both phones used in the experiments are in the upright position with the tester making slight motions throughout the test period. In the *NoMotion* case, both phones are laid flat facing the ceiling with no motions in the video.

In Table 11 we compare the performance and power characteristics of the two cases under 4 different system configurations. Given that the FPS values of WeChat generally have considerable variations, the results in Table 11

TABLE 12
Performance and Energy Results by Controller: WeChat *NoMotion* with *Normal* Profiling Data

Def Perf	Perf	Perf%	Def Pwr	Pwr	Engy%
17.89	17.60	-1.6%	3732.96	3267.52	-12.5%

Target: 18 FPS.

TABLE 13
Different Loads

Loads	Background Apps
LowLoad (LL)	None
BaseLoad (BL)	Facebook, Yahoo Mail
HighLoad (HL)	Amazon, AngryBirds, Chrome, Ebook Reader Facebook, Skype, Yahoo Mail, Zillow
SyntheticLoad (SL)	Facebook, Yahoo Mail, MB_500_10

indicate that the two cases are rather similar despite that in one case the video is motionless.

To test the impact of the level of motion in the video on the controller, we tested *NoMotion* using the profiling data for *Normal*, and the results are shown in Table 12. Not surprisingly, results close to those of controlling *Normal* with its own profiling data are achieved.

We thus conclude that for WeChat Video, our approach is robust when the level of motion in the video varies.

5.3 Effect of Background Load

In the experiments in Section 5.1, we collected the profiling data for the apps under Base Load (BL), and then ran the controller in the same load condition and achieved good results. However, a random user is likely to have a different background load from our Base Load where the profiling data are collected. In this section, we investigate the effect of background loads on the profiling data and thus on the effectiveness of the controller. The research question we try to answer is the following.

RQ2. How robust are the profiling data and the general approach when subjected to different background loads?

To study this problem we first select a few loads that are different from BL. We then test the controller under each load using the profiling data from the same load. We refer to such data as *native profiling data*. This would show whether or not the load condition could affect the effectiveness of the controller. Following this, we test the controller under each load using the profiling data from BL to examine whether the profiling data from the Base Load are robust enough to be used in a different load condition.

5.3.1 Background Load Characteristics

We selected three additional loads for comparison and studied their impact on the controller. A load is basically a collection of apps running in the background. Table 13 lists the selected loads along with BL, and their constituent apps. In all cases WiFi is turned on. While LL and HL are straightforward, the synthetic load SL consists of running *MB_500_10*, a CPU-intensive micro-benchmark we created, on top of BL.

We start by comparing the resource usage of the different loads when the phone is idling, i.e., no app running in the

TABLE 14
Resource Usage (Average, Median, 90-Percentile), Num. of Threads, Active Memory, and Power of Different Loads

Load	CPU util (%)	Mem Acc./s ($\times 1024$)	IPS ($\times 10^9$)	Threads	Mem (MB)	Power (mW)
LowLoad (LL)	2.06, 1.44, 4.56	994, 518, 2154	0.0283, 0.0176, 0.0548	737	264	267.47
BaseLoad (BL)	2.03, 1.47, 3.89	1021, 579, 2122	0.0293, 0.0169, 0.0569	917	353	265.57
HighLoad (HL)	2.14, 1.93, 3.60	1128, 589, 2715	0.0316, 0.0173, 0.0747	1425	935	277.46
SyntheticLoad (SL)	14.28, 13.87, 16.27	1740, 1237, 3609	0.2482, 0.2366, 0.2922	946	366	374.22

foreground. In Table 14 we show for each load the CPU utilization, memory access rate, instructions per second, as well as the number of schedulable threads, the amount of active memory and power consumption.

It can be seen that, in terms of CPU utilization, memory access rate, IPS, and power consumption, LL and HL are almost indistinguishable from BL. Intuitively, since HL runs many more apps, the majority of which are resource-hungry, than LL, much more resource should be used along with noticeably higher power. That, however, is not the case. The biggest difference for these three loads is in the number of threads and memory usage. The SL, on the other hand, is quite different from the others. Its CPU utilization, memory access, and IPS are much higher. It also consumes about 100 mW more power.

5.3.2 Testing of Loads with Native Profiling Data

For each of the loads, we first profile the apps, and then test the controller using the profiling data. In other words, this is similar to Section 5.1 in that the runtime load is the same as the load for profiling.

The app profiling data under LL and HL are very similar. Given their similarity in resource usage and power consumption, as shown in Table 14, we expect the controller to have similar results in these two loads. The test results in Table 15 for LL and Table 16 for HL indeed confirm that. Compare these two tables with the results for BL in Section 5.1, we can see in fact they are all very similar.

Despite its different performance and power characteristics from other loads, the SL produced similar results as well, which are shown in Table 17. The variations of performance seem to be greater than the other loads. The energy savings, however, are very close to those in the other loads.

To summarize, we show in Fig. 11a side-by-side comparison of the controller's performance and energy results in

all the four loads using native profiling data. Except the WeChat performance results, these results are quite similar across the loads, indicating that our approach is highly robust across different background loads.

5.3.3 Testing with Base Load Profiling Data

The previous section demonstrates that the controller produces good results across different loads when using native profiling data. In this section we examine how the controller performs when using the BL profiling data in different loads.

For LL and HL, we expect the controller to perform well with BL data because (1) the BL data are very close to those of LL and HL, and (2) the load characteristics of these loads, as shown in Table 14, are very close. Results in Table 18 for LL and Table 19 for HL confirm this.

The SyntheticLoad, however, has very different load characteristics. We therefore expect profiling data in this load to be quite different from those in BL. However, as it turns out, the profiling difference is not as great as we expected. Fig. 12 shows the profiling data in SL normalized to the BL data. For each app, four configurations are shown with the first one in each group representing the case for default governors. In terms of performance, WeChat Video appears to show

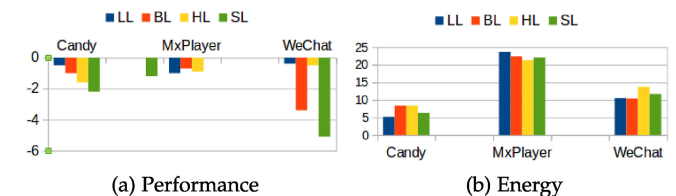


Fig. 11. Controller performance difference and energy savings: all loads using native profiling data.

TABLE 15
Performance and Energy Results: Low Load

App	Def Perf	Perf	Perf%	Def Pwr	Pwr	Engy%
Candy	55.60	55.34	-0.5%	2626.96	2491.59	-5.2%
MxPlayer	23.97	23.74	-1.0%	3316.04	2530.72	-23.7%
WeChat	18.13	18.05	-0.4%	3737.02	3345.07	-10.5%

Target FPS: 56 (Candy), 23.9 (MxPlayer), and 18 (WeChat).

TABLE 16
Performance and Energy Results: High Load

App	Def Perf	Perf	Perf%	Def Pwr	Pwr	Engy%
Candy	56.16	55.26	-1.6%	2680.42	2454.78	-8.4%
MxPlayer	23.94	23.72	-0.9%	3262.79	2567.39	-21.3%
WeChat	18.15	18.06	-0.5%	3894.31	3359.32	-13.7%

Target FPS: 56 (Candy), 23.9 (MxPlayer), and 18 (WeChat).

TABLE 17
Performance and Energy Results: Synthetic Load

App	Def Perf	Perf	Perf%	Def Pwr	Pwr	Engy%
Candy	57.45	56.19	-2.2%	2795.44	2618.50	-6.3%
MxPlayer	23.96	23.67	-1.2%	3367.08	2622.53	-22.1%
WeChat	16.78	15.93	-5.1%	3960.64	3496.54	-11.7%

Target FPS: 56 (Candy), 23.9 (MxPlayer), and 18 (WeChat).

TABLE 18
Performance and Energy Results by Controller Under LL
Using Profiling Data From BL

App	Def Perf	Perf	Perf%	Def Pwr	Pwr	Engy%
Candy	55.60	55.76	0.3%	2626.96	2484.90	-6.2%
MxPlayer	23.97	23.86	-0.5%	3316.04	2535.24	-23.5%
WeChat	18.13	17.59	-3.0%	3737.02	3306.81	-11.5%

Target FPS: 56 (Candy), 23.9 (MxPlayer), and 18 (WeChat).

TABLE 19
Performance and Energy Results by Controller Under HL
using Profiling Data from BL

App	Def Perf	Perf	Perf%	Def Pwr	Pwr	Engy%
Candy	56.16	55.25	-1.6%	2648.80	2488.90	-6.0%
MxPlayer	23.94	23.91	-0.1%	3262.79	2537.40	-22.2%
WeChat	18.56	17.95	-3.3%	3869.98	3333.79	-13.8%

Target FPS: 56 (Candy), 23.9 (MxPlayer), and 18 (WeChat).

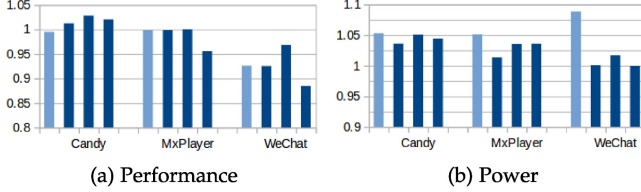


Fig. 12. Profiling data under SyntheticLoad normalized to BaseLoad data. Four configurations shown for each app, the first in each group being the default case.

the biggest differences with one configuration having a difference of more than 10 percent. Another noticeable difference is the power value of WeChat with the default governors. Other than those, the differences between the two loads are generally less than 5 percent. We therefore expect our controller to perform well.

Table 20 shows the controller test results in SL using the profiling data from BL. Compare these results with Table 17 where the native profiling data were used, we can see our controller handles the load difference very well.

5.4 Summary

Experimental results in this section show that the system is robust when subjected to variations in inputs and background loads. This robustness is largely due to the adaptation of the per-configuration performance prediction model, which allows the system to independently estimate the app performance for each configuration, as shown in Fig. 10. Once the performance estimate is accurate, our feedback controller is able to meet performance target while saving energy. The robustness of the system also benefits from the fact that random factors such as user inputs and background loads do not distort profiling data significantly, allowing profiling data obtained in one setting to be used in another.

6 CONCLUSIONS

Addressing the energy inefficiency problem of DVFS governors found on current Android device, this paper proposes an application-specific approach to energy optimization. Our approach consists of offline app profiling and online controlling. First, we profile the target app by measuring its performance and power consumption under various DVFS configurations. Then the profiling data are passed on to the online controller which uses feedback control to select energy-optimal DVFS settings for the app while trying to meet a performance target. Contrary to the default governors, our method uses application-specific data, manages energy, and controls CPU and memory simultaneously. In this paper we present detailed description of the system design and experimental evaluation with selected mainstream apps. In particular, we focus on two problems

TABLE 20
Performance and Energy Results by Controller Under SL
Using Profiling Data From BL

App	Def Perf	Perf	Perf%	Def Pwr	Pwr	Engy%
Candy	57.45	55.65	-3.1%	2795.44	2623.06	-6.2%
MxPlayer	23.96	23.79	-0.7%	3367.08	2627.85	-21.9%
WeChat	16.78	16.14	-3.8%	3960.64	3472.49	-12.3%

Target FPS: 56 (Candy), 23.9 (MxPlayer), and 18 (WeChat).

related to the robustness of the system, namely, the impact of input dependence and background load. Experiments show that our approach is robust enough to handle these types of variations.

REFERENCES

- [1] Number of smartphone users worldwide from 2014 to 2020 (in billions), [Online]. Available: <https://www.statista.com>
- [2] F. Schlachter, "No moore's law for batteries," *Proc. Nat. Acad. Sci. United States of America*, vol. 110, no. 14, pp. 5273–5273, 2013.
- [3] M. Weiser, B. Welch, A. Demers, and S. Shenker, "Scheduling for reduced CPU energy," in *Mobile Computing*, Norwell, MA, USA: Kluwer Academic Publishers, 1994, pp. 449–471.
- [4] P. Pillai and K. G. Shin, "Real-time dynamic voltage scaling for low-power embedded operating systems," *ACM SIGOPS Operating Syst. Rev.*, vol. 35, no. 5, pp. 89–102, 2001.
- [5] V. Pallipadi and A. Starikovskiy, "The ondemand governor," in *Proc. Linux Symp.*, 2006, vol. 2, pp. 215–230.
- [6] K. Rao, J. Wang, S. Yalamanchili, Y. Wardi, and H. Ye, "Application-specific performance-aware energy optimization on Android mobile devices," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit.*, 2017, pp. 169–180.
- [7] CPU frequency and voltage scaling code in the linux kernel, [Online]. Available: <https://goo.gl/Mlr9U1>
- [8] H. David, C. Fallin, E. Gorbato, U. R. Hanebutte, and O. Mutlu, "Memory power management via dynamic voltage/frequency scaling," in *Proc. 8th ACM Int. Conf. Auton. Comput.*, 2011, pp. 31–40.
- [9] Linux kernel 3.2 release notes, [Online]. Available: https://kernelnewbies.org/Linux_3.2
- [10] P. Bezerra, L. Araujo, G. B. Ribeiro, A. Neto, A. Silva-Filho, C. Siebra, F. Q. B. da Silva, A. Santos, A. Mascaro, and P. Costa, "Dynamic frequency scaling on android platforms for energy consumption reduction," in *Proc. 8th ACM Workshop Perform. Monitoring Meas. Heterogeneous Wireless Wired Netw.*, 2013, pp. 189–196.
- [11] Wechat, [Online]. Available: <https://play.google.com/store/apps/details?id=com.tencent.mm>
- [12] C. Ellis, "The case for higher-level power management," in *Proc. Workshop Hot Topics Operating Syst.*, 1999, pp. 162–167.
- [13] Y. Zhu, M. Halpern, and V. J. Reddi, "Event-based scheduling for energy-efficient QoS (eQoS) in mobile web applications," in *Proc. IEEE 21st Int. Symp. High Perform. Comput. Archit.*, 2015, pp. 137–149.
- [14] D. H. Bui, Y. Liu, H. Kim, I. Shin, and F. Zhao, "Rethinking energy-performance trade-off in mobile web page loading," in *Proc. 21st Annu. Int. Conf. Mobile Comput. Netw.*, 2015, pp. 14–26.
- [15] N. C. Nachiappan, P. Yedlapalli, N. Soundararajan, A. Sivasubramaniam, M. T. Kandemir, R. Iyer, and C. R. Das, "Domain knowledge based energy management in handhelds," in *Proc. IEEE 21st Int. Symp. High Perform. Comput. Archit.*, 2015, pp. 150–160.
- [16] W.-Y. Liang, M.-F. Chang, Y.-L. Chen, and C.-F. Lai, "Energy efficient video decoding for the android operating system," in *Proc. IEEE Int. Conf. Consumer Electron.*, 2013, pp. 344–345.
- [17] L. Li, J. Wang, X. Wang, H. Ye, and Z. Hu, "SceneMan: Bridging mobile apps with system energy manager via scenario notification," in *Proc. ACM/IEEE Int. Symp. Low Power Electron. Des.*, 2017, pp. 1–6.
- [18] Q. Deng, D. Meisner, A. Bhattacharjee, T. F. Wenisch, and R. Bianchini, "CoScale: Coordinating CPU and memory system DVFS in server systems," in *Proc. 45th Annu. IEEE/ACM Int. Symp. Microarchitecture*, 2012, pp. 143–154.
- [19] R. Begum, D. Werner, M. Hempstead, G. Prasad, and G. Challen, "Energy-performance trade-offs on energy-constrained devices with multi-component DVFS," in *Proc. IEEE Int. Symp. Workload Characterization*, 2015, pp. 34–43.

- [20] A. Pathania, Q. Jiao, A. Prakash, and T. Mitra, "Integrated CPU-GPU power management for 3D mobile games," in *Proc. 51st Annu. Des. Autom. Conf.*, 2014, pp. 1–6.
- [21] A. Carroll and G. Heiser, "Unifying DVFS and offlining in mobile multicores," in *Proc. IEEE 20th Real-Time Embedded Technol. Appl. Symp.*, 2014, pp. 287–296.
- [22] D. Kadjo, R. Ayoub, M. Kishinevsky, and P. V. Gratz, "A control-theoretic approach for energy efficient CPU-GPU subsystem in mobile platforms," in *Proc. 52nd Annu. Des. Autom. Conf.*, 2015, Art. no. 62.
- [23] C. Imes, D. H. Kim, M. Maggio, and H. Hoffmann, "POET: A portable approach to minimizing energy under soft real-time constraints," in *Proc. 21st IEEE Real-Time Embedded Technol. Appl. Symp.*, 2015, pp. 75–86.
- [24] X. Chen, A. Jindal, N. Ding, Y. C. Hu, M. Gupta, and R. Vannithamby, "Smartphone background activities in the wild: Origin, energy drain, and optimization," in *Proc. 21st Annu. Int. Conf. Mobile Comput. Netw.*, 2015, pp. 40–52.
- [25] M. Martins, J. Cappos, and R. Fonseca, "Selectively taming background android apps to improve battery lifetime," in *Proc. USE-NIX Annu. Tech. Conf.*, 2015, pp. 563–575.
- [26] Candy crush saga, [Online]. Available: <https://play.google.com/store/apps/details?id=com.king.candycrushsaga&hl=en>
- [27] G. Welch and G. Bishop, "An introduction to kalman filter," Univ. North Carolina at Chapel Hill, Chapel Hill, NC, Tech. Rep. TR 95–041, 1995.
- [28] K. J. Aström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton, NJ, USA: Princeton Univ. Press, 2010.
- [29] B. Kolman and R. E. Beck, *Elementary Linear Programming with Applications*, 2nd ed. Cambridge, MA, USA: Academic Press, 1995.
- [30] P. Mochel, "The SYSFS filesystem," in *Proc. Linux Symp.*, 2005, pp. 313–326.
- [31] Monsoon power monitor, [Online]. Available: <https://www.monsoon.com/LabEquipment/PowerMonitor/>
- [32] Mx player, [Online]. Available: <https://play.google.com/store/apps/details?id=com.mxtech.videoplayer.ad&hl=en>
- [33] Number of monthly active wechat users from 2nd quarter 2010 to 2nd quarter 2017 (in millions), [Online]. Available: <https://www.statista.com>
- [34] Quicktime file format, [Online]. Available: https://en.wikipedia.org/wiki/QuickTime_File_Format
- [35] Video file format, [Online]. Available: https://en.wikipedia.org/wiki/Video_file_format



Jun Wang received the PhD degree in computer science from McGill University. He is a senior researcher at Huawei US R&D, in Santa Clara, CA, where he has been leading research projects on optimizing energy consumption for Android mobile devices. His research interests include energy optimization for mobile devices, compiler, computer architecture, parallel simulation, distributed computing, among others. He is a member of the IEEE Computer Society.



Karthik Rao received the bachelor's degree in electronics and communication engineering from the Manipal Institute of Technology, India, and the master's and PhD degrees from Georgia Tech, Atlanta, with a focus on control theory and computer architecture. His technical interest areas are application of feedback control in computing systems, optimization, multi-agent dynamical systems, machine learning, modeling, and simulation architectures.



Handong Ye received the PhD degree from Tsinghua University. He is a director at Huawei US R&D, in Santa Clara, CA, where he oversees R&D projects in such areas as optimizing compiler, programming language and runtime, and energy optimization for mobile devices. In addition to the above, his technical interests also include virtual machine, OS, mobile, and cloud computing.

► For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.